



Unlocking Silent Signals: Decoding Body Language with Mediapipe

Nachiketa Venkat (23BCE0461) &
Sarthak Gupta (23BDS0047)

Unlocking Silent Signals: Decoding Body Language with Mediapipe

❖ Project Description:

"Body Language Decoder Using Mediapipe" is a project that leverages Google's Mediapipe framework to analyze and interpret body language signals. Mediapipe's machine learning and pose estimation capabilities are used to identify and decode nonverbal cues such as gestures, facial expressions, and body posture. The goal of the project is to provide users with insights into social interactions and communication styles based on body language analysis. This can be particularly useful in fields such as human-computer interaction, social robotics, psychology, and professional communication coaching.

i. Scenario 1: Human-Computer Interaction

By integrating the body language decoder into interactive systems, applications can become more responsive to user emotions and intentions. For example, virtual assistants or smart devices can adjust their responses based on a user's gestures or expressions, creating a more natural and intuitive interaction experience.

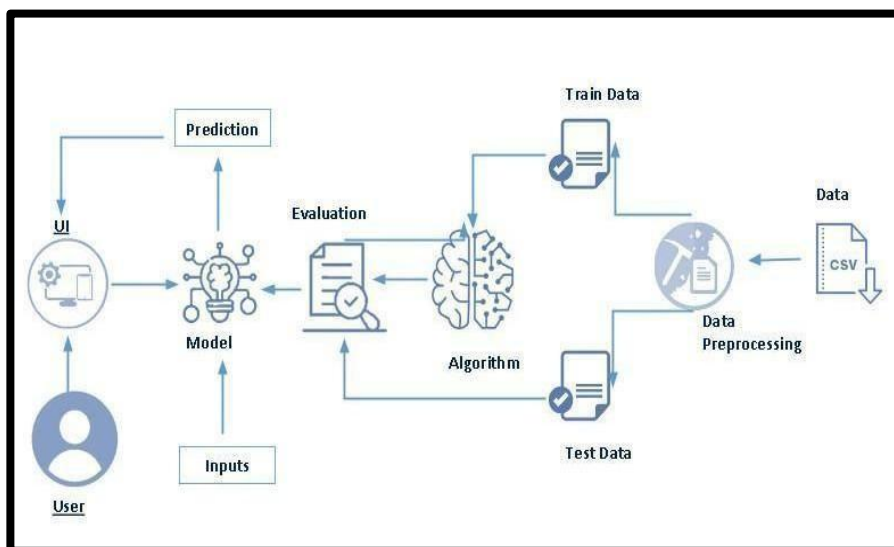
ii. Scenario 2: Communication Coaching

The body language decoder can be used by professional coaches to help individuals improve their communication skills. By analyzing nonverbal cues during presentations or meetings, coaches can provide targeted feedback on body posture, gestures, and facial expressions to enhance an individual's overall effectiveness and confidence.

iii. Scenario 3: Social Robotics

In the field of social robotics, the body language decoder can be applied to enable robots to interpret human nonverbal signals and respond appropriately. This capability can enhance human-robot interactions, allowing robots to better understand and react to human emotions and social cues, making them more empathetic and relatable.

Technical Architecture:



Project Flow

- *The user interacts with the UI to upload the file.*
- *Uploaded input is analyzed by the model which is integrated/developed by you.*
- *Once the model analyzes the input, the prediction is showcased on the UI. To accomplish this, we have to complete all the activities listed below,*

- *Define Problem / Problem Understanding*

0 Specify the business problem

- *Business requirements*
- *Literature Survey.*
- *Social or Business Impact.*

- *Data Collection & Preparation*

- *Collect the dataset*
- *Data Preparation*

- *Exploratory Data Analysis*

0 Descriptive statistical

- *Visual Analysis*

- *Model Building*

- *Building a model.*

0 Training the model.

- *Testing the model*

- *Model Deployment*

0 Save the best model

- *Integrate with Web Framework*

- *Project Demonstration & Documentation*

0 Record explanation Video for project end to end solution

- *Project Documentation-Step by step project development procedure*

Prior Knowledge:

To complete this project, you must require following software's , concepts and packages

- *VS Code :*
- *Refer to the link below to download VS Code.*
- *Link : <https://code.visualstudio.com/download>*
- *Create Project:*
- *Open VS Code.*
- *Create a Folder and name it "AI_Body_Language_Detector".*
- *Machine Learning Concepts*
- *Machine Learning:*

<https://towardsdatascience.com/machine-learning-an-introduction-23b84d51e6d0>

- *ML Models :*

<https://towardsdatascience.com/all-machine-learning-models-explained-in-6-minutes-9fe30ff6776a>

- *Web concepts:*

o Get the gist on streamlit :

<https://www.geeksforgeeks.org/a-beginners-guide-to-streamlit/>

- *Mediapipe:*

o About Mediapipe :

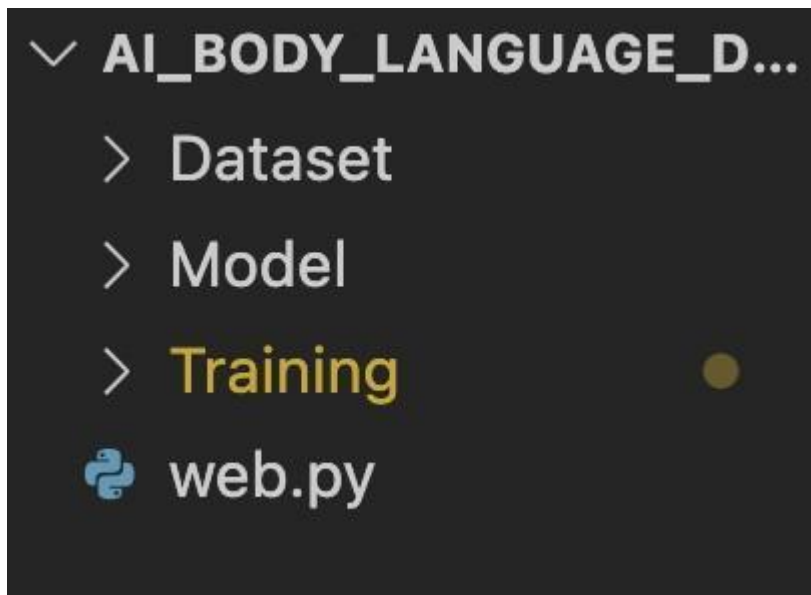
<https://www.geeksforgeeks.org/python-facial-and-hand-recognition-using-mediapipe-holistic/>

Project Objectives:

By the end of this project you will:

- ✓ *Know fundamental concepts and techniques of Machine Learning.*
- ✓ *Gain a broad understanding on Mediapipe.*
- ✓ *Know how to pre-process/clean the data using different data preprocessing techniques.*
- ✓ *Know how to build a web application using the Streamlit framework*

Project Structure:



- ✚ *We are building a Streamlit application which needs a python script web.py for a website.*
- ✚ *Model folder contains your saved models.*
- ✚ *The Training folder contains the code for building/training/testing the model.*
- ✚ *The Dataset folder contains a .csv file created by you.*

Data Collection & Preparation:

ML depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to create the required dataset.

1. Make Some Detections.

First we will import some required packages

```
import mediapipe as mp
```

```
import cv2
```

The MediaPipe Holistic Landmarker task lets you combine components of the pose, face, and hand landmarks to create a complete landmarker for the human body.

Let us initialize the holistics

```
mp_drawing = mp.solutions.drawing_utils #Drawing Helpers  
mp_holistic = mp.solutions.holistic #Mediapipe Solutions
```

Now we'll make some detections of face, hands and pose.

This Python script uses OpenCV and MediaPipe Holistic to capture webcam video and perform real-time detection and visualization of facial landmarks, hand landmarks, and body pose landmarks.

Key Components:

- **Video Capture:**
The script starts by initializing the webcam using `cv2.VideoCapture(0)` to continuously capture frames from the default camera.
 - **Holistic Model Initialization:**
It uses MediaPipe's Holistic model (`mp_holistic.Holistic`) with specified detection and tracking confidence thresholds to detect face, hand, and pose landmarks in each video frame.
 - **Frame Processing Loop:**
The main loop runs as long as the webcam is open:
 - Each frame is read and converted from BGR to RGB color space, as required by MediaPipe.
 - The frame is processed by the Holistic model to get detection results for face, hands, and pose landmarks.
 - The frame is converted back to BGR for OpenCV rendering.
 - **Landmark Visualization:**
The script draws detected landmarks on the frame:
 - Face Landmarks: Drawn with custom colors and thickness.
 - Right Hand Landmarks: Drawn with different colors and larger circles.
 - Left Hand Landmarks: Drawn with another set of colors and circle sizes.
 - Pose Landmarks: Drawn with distinct colors and thicker lines.
 - **Display and Exit:**
The processed frame, with all landmarks drawn, is displayed in a window titled "Raw Webcam Feed". The loop continues until the user presses the 'q' key, at which point the video capture is released and all OpenCV windows are closed.
- Purpose:**
This code provides a real-time demonstration of multi-part human landmark detection (face, hands, pose) using a webcam, visually overlaying detected keypoints and connections on the video stream for applications in gesture recognition, fitness tracking, or interactive systems.

1.1: Collecting data for Happy mood:

Initialize a variable "class_name" equals "Happy".

```
class_name = "Happy"
```

Now let us write a code which allows us to detect our body landmarks and save them into a .csv file.

- + For now we'll collect the data for happy mode. So, once you run the code your webcam will get activated and you've to record your happy face for 10-15 seconds and close the window.*
- + This Python script leverages OpenCV and MediaPipe Holistic to perform real-time detection, visualization, and export of human body landmarks from a webcam feed. The code is structured to capture video, process each frame for holistic landmarks (face, hands, pose), visualize these landmarks, and export the coordinates to a CSV file for further analysis or machine learning applications.*

Key Functionalities:

+ Video Capture Initialization:

The script begins by capturing video from the default webcam using cv2.VideoCapture(0).

+ MediaPipe Holistic Model:

The Holistic model is initialized with detection and tracking confidence thresholds set to 0.5. This model is capable of detecting face, left hand, right hand, and pose landmarks simultaneously.

+ Frame Processing Loop:

The main loop runs as long as the webcam is open:

- Each frame is read and converted from BGR to RGB for compatibility with MediaPipe.*
- The frame is marked as non-writable to optimize processing.*
- The Holistic model processes the frame to detect landmarks.*
- The frame is converted back to BGR for OpenCV visualization.*

+ Landmark Drawing:

The script draws detected landmarks on the video frame with custom colors and thickness for each type:

- Face Landmarks are drawn with blue and green colors.*

- *Right Hand Landmarks use orange and blue.*
- *Left Hand Landmarks use purple and blue.*
- *Pose Landmarks are highlighted with yellow and pink.*

Landmark Coordinate Export:

The script extracts the coordinates (x, y, z, visibility) of pose and face landmarks, flattens them into a single row, and appends a class label (presumably for supervised learning). These rows are written to a CSV file (coords.csv) in append mode, enabling dataset creation for body language or gesture recognition tasks.

Display and Exit:

The processed frame is displayed in a window titled "Raw Webcam Feed". The loop continues until the 'q' key is pressed, after which resources are released and windows are closed.

- **Error Handling:**

A try-except block ensures that the script continues running even if an error occurs during processing or file writing.

- **Applications:**

This code is suitable for:

 *Real-time gesture or pose recognition systems.*

 *Data collection for training machine learning models in body language or action recognition.*

 *Interactive applications requiring live feedback of body movements.*

1.2: Collecting data for Sad mood:

Initialize a variable "class_name" equals "Sad".

```
class_name = "Sad"
```

Now let us write a code which allows us to detect our body landmarks and

save them into a .csv file.

For now we'll collect the data for sad mode. So, once you run the code your webcam will get activated and you've to record your sad face for 10-15 seconds and close the window.

- This Python script utilizes OpenCV and MediaPipe Holistic to perform real-time detection, visualization, and export of human body landmarks from a webcam feed.
- Main Components and Workflow:

+ Video Capture Setup:

The script starts by initializing the webcam using `cv2.VideoCapture(0)`, allowing it to capture live video frames.

+ Holistic Model Initialization:

MediaPipe's Holistic model is set up with a minimum detection and tracking confidence of 0.5. This model can simultaneously detect face, hand, and pose landmarks.

+ Frame Processing Loop:

The script enters a while loop that continues as long as the webcam is open:

- Each frame is read from the webcam.
- The frame is converted from BGR to RGB color space, as required by MediaPipe.
- The frame is marked as non-writable to optimize processing speed.
- The Holistic model processes the frame, detecting landmarks for the face, both hands, and the body pose.

+ Landmark Visualization:

Detected landmarks are drawn on the frame using custom colors and thicknesses for each type:

- Face Landmarks: Blue and teal colors with different thicknesses and circle radii.
- Right Hand Landmarks: Orange and purple.
- Left Hand Landmarks: Purple and blue.
- Pose Landmarks: Yellow and blue.

+ Exporting Landmark Coordinates:

Inside a try-except block (for error resilience), the script:

- Extracts the coordinates (x, y, z, and visibility) for each pose and face landmark.

- *Flattens these into a single list.*
- *Appends a class name (presumably defined elsewhere in the code).*
- *Writes this data as a row into a CSV file (coords.csv) using Python's csv module. This is useful for creating datasets for machine learning tasks such as gesture or pose classification.*

Display and Exit:

The annotated frame is displayed in a window titled "Raw Webcam Feed". The loop continues until the user presses the 'q' key, at which point the video capture is released and all OpenCV windows are closed.

1.3: Collecting data for Victorious mood:

Initialize a variable "class_name" equals "Victorious".

```
class_name = "Victorious"
```

Now let us write a code which allows us to detect our body landmarks and save them into a .csv file.

For now we'll collect the data for victorious mode. So, once you run the code your webcam will get activated and you've to record your victorious body language for 10-15 seconds and close the window.

Victorious Body Language : Put your two hands up and hold your fists.

- *This Python code uses OpenCV and MediaPipe Holistic to perform real-time detection and visualization of human face, hand, and body pose landmarks from a webcam feed. It starts by capturing video frames from the default webcam and initializing the Holistic model, which is capable of detecting 468 face landmarks, 21 hand landmarks per hand, and 33 pose landmarks per frame²⁴. Each frame is processed: it is converted from BGR to RGB for MediaPipe compatibility, then passed through the Holistic model to detect landmarks. The detected landmarks for the face, right hand, left hand, and body pose are drawn on the video frame with different colors and styles for clarity.*
- *Additionally, the script extracts the coordinates (x, y, z, visibility) of pose and face landmarks, flattens them, appends a class label, and writes this data to a CSV file. This allows for the creation of labeled*

datasets for machine learning or gesture recognition tasks³. The process repeats in real time, displaying the annotated video feed until the user presses 'q' to exit. The code is robust, using a try-except block to handle errors during data export, and is well-suited for applications in gesture recognition, fitness tracking, or interactive systems⁴.

1.4: Collecting data for Fight mood:

Initialize a variable "class_name" equals "Fight".

```
class_name = "Fight"
```

Now let us write a code which allows us to detect our body landmarks and save them into a .csv file.

For now we'll collect the data for fight mode. So, once you run the code your webcam will get activated and you've to record your fight body language for 10-15 seconds and close the window.

Fight Body Language : Put your two hands in front of your face and hold your fists.

- This Python code leverages OpenCV and MediaPipe Holistic to perform real-time detection, visualization, and export of human body landmarks from a webcam stream. The script is structured to capture video, process each frame for holistic landmarks (face, hands, pose), visualize these landmarks, and export the coordinates to a CSV file for further analysis or machine learning.
- Main Components and Workflow:

Video Capture Initialization:

The code starts by opening the default webcam using `cv2.VideoCapture(0)`, enabling the capture of live video frames.

Holistic Model Setup:

MediaPipe's Holistic model is initialized with a minimum detection and tracking

confidence of 0.5. This model is capable of detecting face, left hand, right hand, and pose landmarks simultaneously.

Frame Processing Loop:

The main loop (`while cap.isOpened()`) continuously processes each video frame:

- **Frame Acquisition and Color Conversion:**
Each frame is read and converted from BGR to RGB, as required by MediaPipe.
- **Landmark Detection:**
The frame is processed by the Holistic model to detect face, hand, and pose landmarks. The results are stored in the results object.
- **Recoloring for Display:**
The frame is converted back to BGR for OpenCV visualization.

Landmark Visualization:

The script draws detected landmarks on the frame using custom colors and thickness for clarity:

- **Face Landmarks:**
Drawn with two different colors for enhanced visibility.
- **Right and Left Hand Landmarks:**
Each hand's landmarks are drawn with distinct colors and circle radii to differentiate them.
- **Pose Landmarks:**
Pose connections are highlighted with unique colors and thicker lines.

Exporting Landmark Coordinates:

Inside a try-except block (for error resilience), the script:

- **Extracts Coordinates:**
The coordinates (`x, y, z, visibility`) of each pose and face landmark are extracted and flattened into single lists.
- **Concatenates and Labels Data:**
The pose and face data are combined, and a class name is inserted at the beginning of the row.
- **Writes to CSV:**
The data row is appended to a CSV file (`coords.csv`) using Python's `csv` module. This is useful for building datasets for machine learning tasks such as gesture or pose classification.

Display and Exit:

The annotated frame is displayed in a window titled "Raw Webcam Feed". The loop continues until the user presses the 'q' key, after which the video capture is released and all OpenCV windows are closed.

1.5: Read the Collected Data:

Importing required packages :

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

Read the recorded .csv file :



- *Paste the path of the .csv file in the quotes.*

```
df = pd.read_csv('/Users/rishi/BTECH/Programming/My_Projects/AI_Body_Language_Detector/Dataset/coords.csv')
```

- *After this you can just run the following commands to view the data :*

 *“df.head() ”*

 *“df.tail() ”*

2: Data Preparation:

As we have understood how the data is, let's pre-process the collected data.

Divide the data into dependent and independent variable

```
X = df.drop('class', axis=1)
y = df['class']
```

Exploratory Data Analysis:

1) Descriptive statistical:

 *Descriptive analysis is to study the basic features of data with the statistical process. With this describe function we can understand the unique, top and frequent values of categorical features. And we can find mean, std, min, max and percentile values of continuous features.*

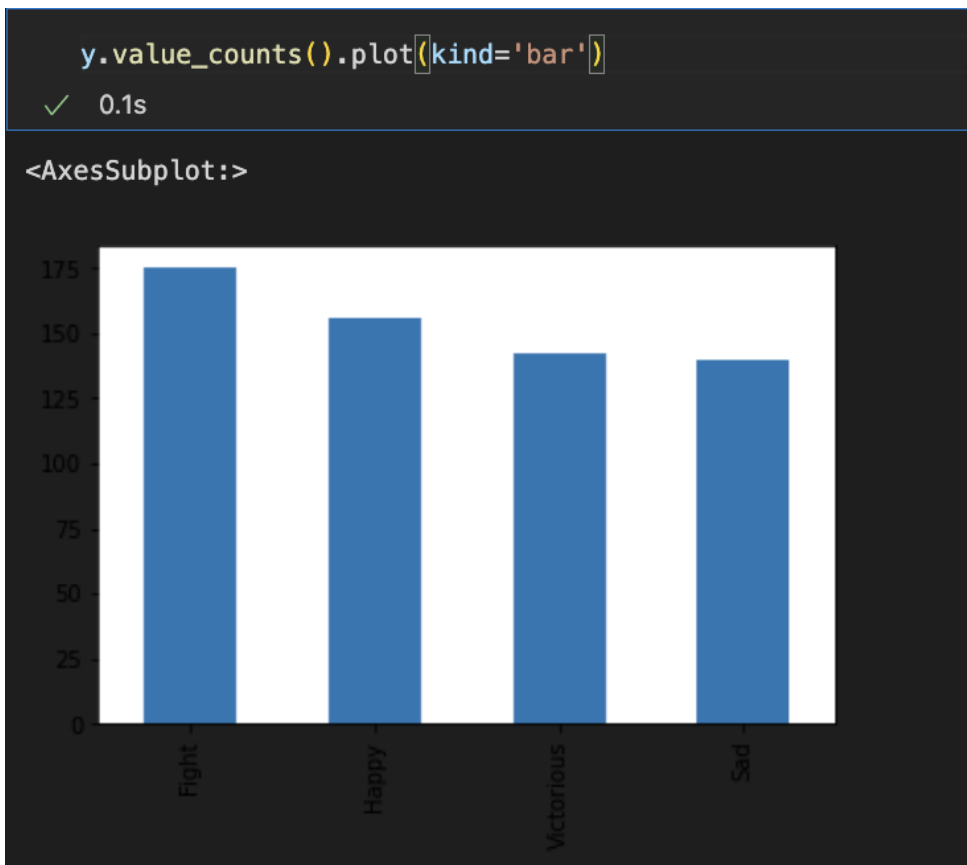
 *Which are not suitable for our dataset.*



- ✚ As our data consists of only coordinates, There is no unnecessary data which can be eliminated.

2) Visual Analysis:

- ✚ Visual analysis is the process of using visual representations, such as charts, plots, and graphs, to explore and understand data. It is a way to quickly identify patterns, trends, and outliers in the data, which can help to gain insights and make informed decisions.
- ✚ Visual analysis is mostly used for numerical data.
- ✚ As we can see data for all the classes are almost equal, so we can proceed with model training
- ✚ If the amount of data is not equal among the categories you can just drop the over recorded rows



3) Splitting data into train and test and validation sets:

✚ Now let's split the Dataset into train and test sets. The split will be in 8:2 ratio : train : test respectively.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1234)
```

Model Building:

1) Importing Necessary Libraries:

✚ We'll import some necessary libraries using the following code

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression, RidgeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

2) Designing a pipeline to train on multiple algorithms:

✚ In this step we are going to create a pipeline which will contain *LogisticRegression*, *RidgeClassifier*, *RandomForestClassifier* and

GradientBoostingClassifier algorithms.

```
pipelines = {  
    'lr':make_pipeline(StandardScaler(), LogisticRegression()),  
    'rc':make_pipeline(StandardScaler(), RidgeClassifier()),  
    'rf':make_pipeline(StandardScaler(), RandomForestClassifier()),  
    'gb':make_pipeline(StandardScaler(), GradientBoostingClassifier()),  
}
```

3) Train the models:

- ✚ Once the pipeline is created we fit all the training data to every model in the pipeline by using the following code

```
fit_models = {}  
for algo, pipeline in pipelines.items():  
    model = pipeline.fit(X_train, y_train)  
    fit_models[algo] = model  
✓ 1m 29.7s
```

Model Deployment:

1) Importing packages:

First we will import required packages

```
from sklearn.metrics import accuracy_score # Accuracy metrics  
import pickle
```

2) Model Evaluation:

- ✚ We will just predict all the models on test data and display the accuracies for each model.
- ✚ We will take the best fit model which can be 'lr', 'rf' or 'gb' 'rc' could be the case of over-fitting.

```
for algo, model in fit_models.items():
    yhat = model.predict(X_test)
    print(algo, accuracy_score(y_test, yhat))
```

✓ 0.1s

lr 0.9924812030075187
rc 1.0
rf 0.9924812030075187
gb 0.9774436090225563

3) Save the Model:

This will allow us to save the .pkl file of the best model.

```
with open('/Users/rishi/BTECH/Programming/My_Projects/AI_Body_Language_Detector/Model/body_language.pkl', 'wb') as f:
    pickle.dump(fit_models['lr'], f)
```

✓ 0.0s

4) Testing the Model:

First we have to load the model using pickle

```
with open('/Users/rishi/BTECH/Programming/My_Projects/AI_Body_Language_Detector/Model/body_language.pkl', 'rb') as f:
    model = pickle.load(f)
```

✓ 0.0s



The following code is final output which allows us to predict the user's body language.



Once you run the code your webcam will get activated, the predictions and the confidence of prediction is shown on the screen.



This code snippet is a Python script that uses OpenCV and MediaPipe to perform real-time holistic human landmark detection (face, hands, and pose) from webcam video, and then predicts body language class using a machine learning model1.



The script starts by capturing video from the webcam with `cv2.VideoCapture(0)`.



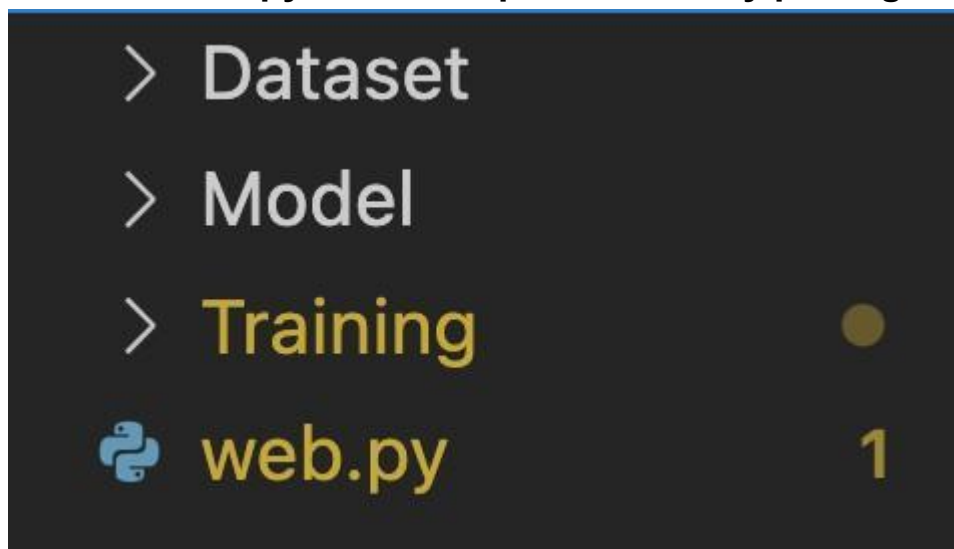
It initializes the MediaPipe Holistic model (`mp_holistic.Holistic`) for detecting face, hand, and pose landmarks, setting minimum detection and tracking confidences to 0.5.

- ✚ Inside the main loop, each frame is read, converted to RGB, and processed by the holistic model to extract landmarks.
- ✚ The script draws detected landmarks for the face, right hand, left hand, and full-body pose on the frame using different colors and drawing specifications for clarity.
- ✚ It then attempts to extract pose and face landmarks, flattens them into a single row, and uses this data as input to a pre-trained machine learning model (`model.predict`) to classify body language and compute prediction probabilities.
- ✚ The predicted class and probability are displayed on the video frame, along with a rectangle and text annotations to highlight the detected region and classification result.
- ✚ The loop continues until the user presses 'q', at which point resources are released and all OpenCV windows are closed.

5) Integrate With Web Framework:

- ✚ *In this section, we will be building a web application that is integrated to the model we built.*
- ✚ *We will be using the streamlit package for our website development.*
- ✚ *Streamlit is a free and open-source framework to rapidly build and share beautiful machine learning and data science web apps.*

6) Create a web.py file and import necessary packages:



```
import streamlit as st
import pandas as pd
import tensorflow as tf
import mediapipe as mp
import cv2
import numpy as np
import pickle
import csv
import os
```

✚ If you get any error like “streamlit not found”. You have to make sure streamlit is installed on your system.

✚ Running the command “pip install streamlit” would do that job for you.

7) Loading the model and creating a frame window to use OpenCV:

```
st.cache(allow_output_mutation=True)

with open('Model/body_language.pkl', 'rb') as f:
    model = pickle.load(f)

st.write("""# Body Language Detection""")
mp_drawing = mp.solutions.drawing_utils #Drawing Helpers
mp_holistic = mp.solutions.holistic #Mediapipe Solutions
run = st.checkbox('Run')
FRAME_WINDOW = st.image([])
camera = cv2.VideoCapture(0)
```

8) Accepting the input from user and prediction

- ✚ This code is a Python script using OpenCV and MediaPipe to perform real-time holistic body, face, and hand landmark detection from a webcam feed, and then classify body language using a machine learning model1.
- ✚ The script starts by initializing a MediaPipe Holistic model with minimum detection and tracking confidence thresholds. It then opens a video capture stream from the webcam and processes each frame in a loop. Each frame is converted to RGB for MediaPipe processing, and then landmark detections are performed for the face, right hand, left hand, and pose (body). The detected landmarks are drawn on the frame using custom colors for each type.
- ✚ The code then attempts to extract pose and face landmarks as flattened arrays. These are combined into a single row, which is used as input for a pre-trained machine learning model to predict the body language class and its probability. The script also calculates the coordinates of the left ear landmark to position the output text and bounding boxes on the frame.
- ✚ The predicted body language class and probability are displayed on the video frame using OpenCV's drawing functions. If the user presses 'q', the loop breaks, and the camera is released and all OpenCV windows are closed. If any error occurs during landmark extraction or prediction, the exception is caught, and the script continues processing the next frame.
- ✚ This script is typically used for applications such as gesture recognition, fitness tracking, or interactive systems that require real-time body language analysis
- ✚ `st.write()` function will write the title for your website.
- ✚ `FRAME_WINDOW.image(image)` will run the OpenCV on the streamlit frame.

To run your website go to your terminal with your respective directory and run the command :

- ✚ `"streamlit run web.py"`

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`

Network URL: `http://192.168.1.11:8501`

For better performance, install the Watchdog module:

```
$ xcode-select --install
```

```
$ pip install watchdog
```

🌈 On successful execution you'll get to see this in your terminal.

HOW DOES YOUR WEBSITE LOOK LIKE ?

🌈 Before clicking on Run checkbox :

Body Language Detection

☐ Run

Stopped

- After clicking on Run :

